



PRACTICAL: 02

Aim: To implement and find the time analysis of Linear Search and Binary Search algorithm.

A) Linear Search:

Algorithm:

Linear Search

input: Array A with n elements and search element key

output: Position of key if found, otherwise
"Element not found".

1. Start
2. Read n, array A and search element key.
3. Set $i = 0$
4. Compare $A[i]$ with key.
5. if $A[i] == \text{key}$, display the position and stop
6. increment i and repeat until $i < n$.
7. if the element is not found, display
"Element not found".
8. Stop

Program / Code:

```
#include <iostream>

using namespace std;

int main() {

    int arr[100],i,n,key;

    cout<<"enter number of array elements:"<<endl;

    cin>>n;

    cout<<"enter array elements:"<<endl;

    for(i=0;i<n;i++){

        cin>>arr[i];

    }

    cout<<"enter element to serach:"<<endl;

    cin>>key;

    for(i=0;i<n;i++){

        if(arr[i]==key)

        {

            cout<<"element found at index of:"<<i;

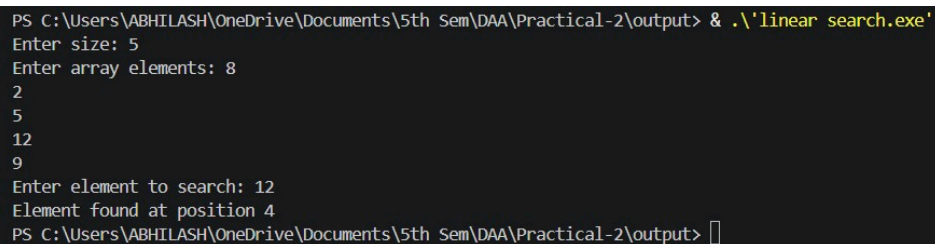
        }

    }

    return 0;

}
```

Output:



```
PS C:\Users\ABHILASH\OneDrive\Documents\5th Sem\DAA\Practical-2\output> & .\'linear search.exe'
Enter size: 5
Enter array elements: 8
2
5
12
9
Enter element to search: 12
Element found at position 4
PS C:\Users\ABHILASH\OneDrive\Documents\5th Sem\DAA\Practical-2\output> █
```



Time Complexity:

Linear Search

Best case : $O(1)$ - element at first

Average case : $O(n)$

worst case : $O(n)$ - element is last or not present

→ Here in worst case it checks n elements

so $T(n) = O(n)$

B) Binary Search:

Algorithm:

Binary Search

input:- Sorted array A with n elements and search element key .

output:- Position of key if found, otherwise "Element not found"

1. Start
2. Read n , sorted array A and search element key .
3. Set $low = 0$ and $high = n - 1$.
4. Repeat while $low \leq high$:
 - set $mid = (low + high) / 2$
 - if $A[mid] == key$, display the position and stop
 - if $key < A[mid]$, set $high = mid - 1$
 - otherwise, set $low = mid + 1$.
5. if the element is not found, display "Element not found".
6. Stop

Program / Code:

```
#include <iostream>

using namespace std;

void bubblesort(int arr[], int n) {
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n-1; while(low <=
    high) {
        int mid = (low + high) / 2; if(arr
        [mid] == key) return mid; else if(arr
        [mid] < key) low = mid+1; else high
        = mid-1;
    }
    return -1;
}

int main() { int arr[100], n, key; cout <<
"Enter number of array elements: ";
```

```
cin >> n;

cout << "Enter array elements: ";
for(int i=0; i<n; i++) {
    cin >> arr[i];
}

bubblesort(arr, n);

cout << "Sorted elements: ";
for(int i=0; i<n; i++) {
    cout << arr[i] << " ";
}

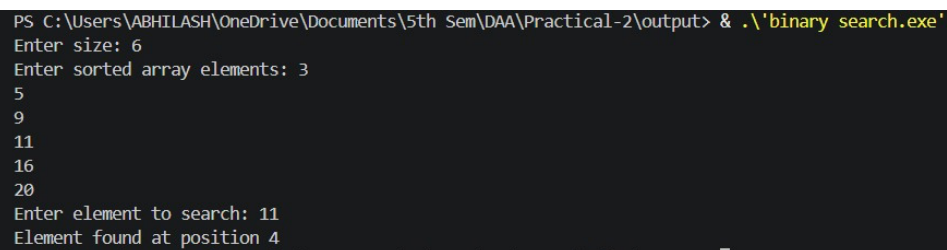
cout << endl;

cout << "Enter element to search: ";
cin >> key;

int result = binarySearch(arr, n, key);
if(result != -1)
    cout << "Element found at index " << result << endl;
else
    cout << "Element not found" << endl;

return 0;
}
```

Output:



```
PS C:\Users\ABHILASH\OneDrive\Documents\5th Sem\DAA\Practical-2\output> & .\'binary search.exe'
Enter size: 6
Enter sorted array elements: 3
5
9
11
16
20
Enter element to search: 11
Element found at position 4
```

Time Complexity:

Binary Search

Binary searches only one half $\rightarrow T(\frac{n}{2})$

The middle element comparison takes constant time $\rightarrow +1$

$$T(n) = T\left(\frac{n}{2}\right) + 1 \quad \text{--- ①}$$

$n \rightarrow \frac{n}{2}$ Put in ①

$$T\left(\frac{n}{2}\right) = T\left(\frac{n}{4}\right) + 1 \quad \text{--- ②}$$

Sub ② in ①

$$T(n) = T\left(\frac{n}{4}\right) + 2 \quad \text{--- ③}$$

$n \rightarrow \frac{n}{4}$ Put in ③

$$T\left(\frac{n}{4}\right) = T\left(\frac{n}{8}\right) + 3$$

$\vdots$

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

$$\frac{n}{2^k} = 1, \quad n = 2^k$$

apply log on b.o.s

$$\log n = \log 2^k$$

$$k = \log n$$

$$T(n) = T\left(\frac{n}{2^{\log n}}\right) + \log n$$

$$= T\left(\frac{n}{n^{\log 2}}\right) + \log n$$

$$= T(1) + \log n$$

$$T(n) = O(\log n) \rightarrow \text{in worst case}$$

$$T(n) = O(1) \rightarrow \text{in Best case}$$

$$T(n) = O(\log n) \rightarrow \text{in Average case.}$$

Conclusion:

In this practical, we studied and implemented two searching algorithms: Linear Search checks each element one by one, so it takes more time as the list size grows. Binary Search is much faster because it works only on sorted lists and cuts the list in half every time, making it a better choice for large data.